Order Matching Engine

Mid-sem Practice Mock 2: EGC 211P Section A

Duration: 120 minutes

Original practice problem at GIE-level logic, in the 2026 Mid-sem format

READ THE INSTRUCTIONS CAREFULLY

Contents of Mock2.zip

```
question.pdf                                          ← this problem statement
Sample_Runs.pdf                          ← end-to-end traces (stdout and log file)
skeleton.cpp                                           ← complete, then submit
local_tester/                                  ← local testcases (do not edit)
```

Write every class yourself in `skeleton.cpp`. It contains only the `Side` enum, a price formatter `fmt2`, the includes and an empty `main`.

Write the classes in the order of the sections: `ExchangeException` → `Logger` → `Order` → `BuyOrder`/`SellOrder` → `Trade` → `OrderBook` → `Exchange`.

Names, parameter types and order, return types and `const` must match the UML exactly. The testcases call your methods directly.

Compilation error ⇒ 0 marks. If some code does not compile, comment it out before submitting.
DO NOT use `bits/stdc++.h`. 50% penalty.

Every message and output line in this pdf is exact. Copy-paste them. Format every price with `fmt2`, never with `to_string`.

Local testcases (in the folder that has `skeleton.cpp`): `sh ./local_tester/local_test.sh`

Stop adding new code 10 minutes before the end.

1 Application Overview

The application is the matching engine of a small stock exchange. Traders register with some cash. They then send *limit orders*: "buy up to q shares of `INFY` paying at most p each", or "sell up to q shares at no less than p each". Each stock symbol has its own *order book*, which holds the orders that are waiting.

Vocabulary (used throughout):

- **bid**: a resting buy order. **ask**: a resting sell order. **Resting** means it is waiting in the book.
- **incoming order**: the order currently being submitted.
- An incoming buy at price p **crosses** an ask at price a if $p \geq a$. An incoming sell at price p crosses a bid at price b if $p \leq b$.
- **remaining**: the quantity of an order not yet traded. An order with remaining $= 0$ is **filled**.

When an order arrives, the engine matches it against the opposite side of the book, repeatedly taking the *best* eligible resting order (Section 9.1). Every match produces a *trade* at the **resting** order's price. Whatever quantity is still unmatched at the end then rests in the book. A trader's orders never trade with each other (*self-trade prevention*). The exchange settles every trade by moving cash and shares between the two traders, and writes each event to a log file.

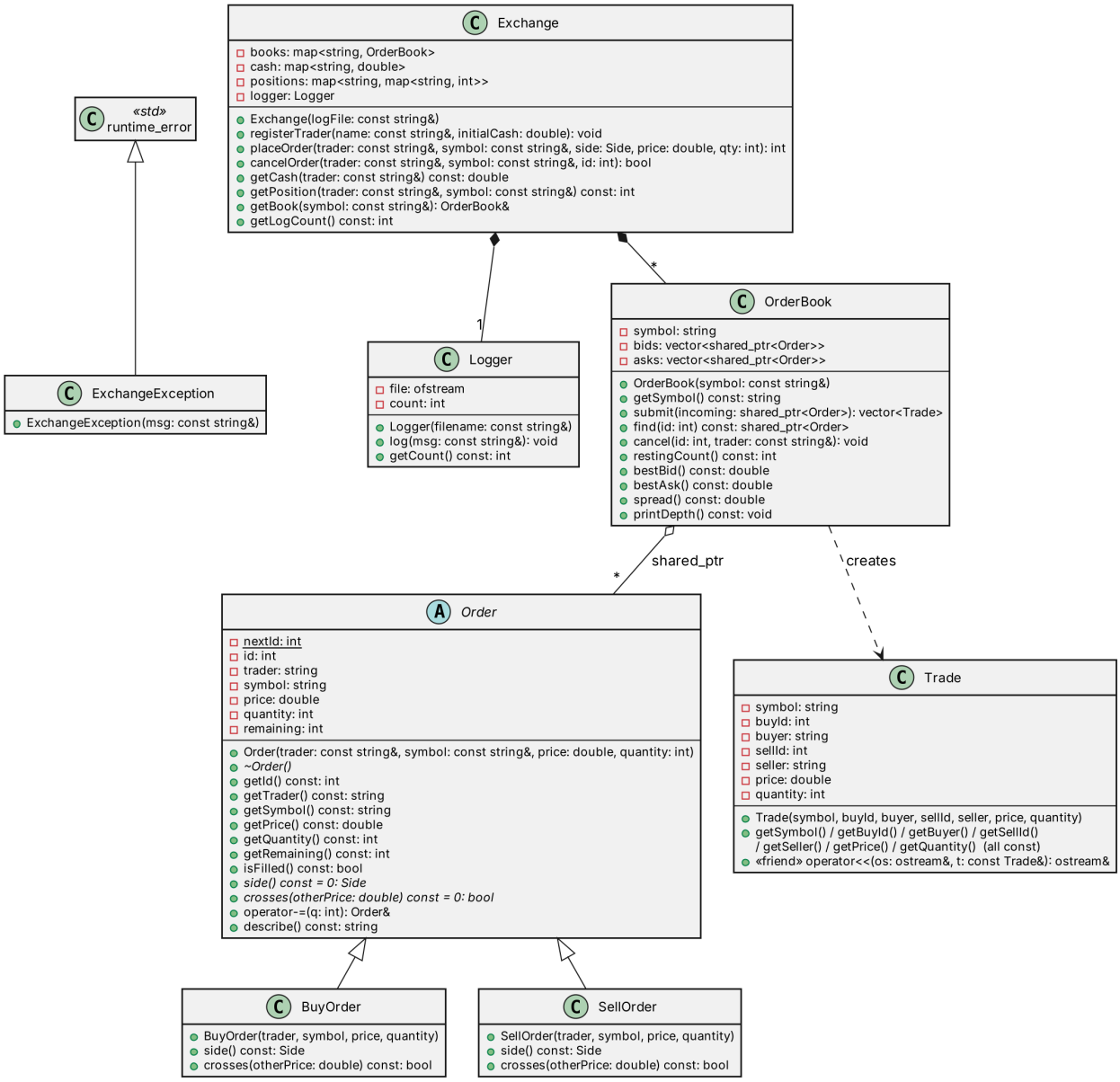
2 Classes

Implement them in this order (each depends on the earlier ones):

1. `ExchangeException`: the single exception type, carrying a message (Section 4).
2. `Logger`: appends lines to a log file (Section 5).
3. `Order`: abstract base of all orders (Section 6).
4. `BuyOrder`, `SellOrder`: the two concrete orders (Section 7).
5. `Trade`: one execution between a buy and a sell (Section 8).
6. `OrderBook`: the book of one symbol and the matching algorithm (Section 9).
7. `Exchange`: traders, books, settlement, logging (Section 10).

Add any private helpers you need. Do not remove or rename anything in the UML.

3 UML Class Diagram



Notation.

red square / green circle	private / public
underlined	static
italic, `= 0`	virtual; `= 0` is pure virtual (abstract class `A`)
hollow triangle	public inheritance
filled diamond	composition: `Exchange` owns its `Logger` and its `OrderBook`s as members
hollow diamond	aggregation via `shared_ptr<Order>`: an order is shared between the book and whoever created it
dashed arrow	dependency: `OrderBook` creates `Trade` objects

Indicative marks. UML translation 15, `ExchangeException` + `Logger` 8, `Order` family 15, `Trade` 5, `OrderBook` 35, `Exchange` 22.

4 ExchangeException Class

Publicly derives from `std::runtime_error`. The constructor passes `msg` to the base constructor, so `what()` returns it. Every error in this problem is an `ExchangeException`. Table 1 lists all the messages.

5 Logger Class

- The constructor opens `filename` in **append** mode (`ios::app`): earlier contents are kept. If the file cannot be opened, `throw runtime_error("Cannot open log file");`. `count` starts at 0.
- `log(msg)` writes `msg` followed by `\n`, flushes the file immediately (`file.flush()`), and increments `count`.
- `getCount()` returns the number of lines this object has logged.

6 Order Class (abstract)

6.1 Constructor and IDs

`nextId` is a private static counter that starts at **1**. The constructor stores the arguments and sets `remaining = quantity`. It then validates, in this order:

1. `quantity <= 0` ⇒ `throw ExchangeException("Invalid quantity");`
2. `price <= 0` ⇒ `throw ExchangeException("Invalid price");`

Only after both checks pass: `id = nextId++;`. **A rejected order must not use up an ID.** If order 1 succeeds, the next one fails validation, and the one after succeeds, the third gets ID 2.

6.2 Methods

- Getters return the members. `isFilled()` is `remaining == 0`.
- `side()` and `crosses()` are pure virtual (Section 7).
- `operator-=(q)` records a fill of `q` shares. If `q <= 0` or `q > remaining`: `throw ExchangeException("Invalid fill");`. Otherwise `remaining -= q`. Returns `*this`.
- `describe()` returns (no newline):

```
#<id> <BUY|SELL> <trader> <symbol> <remaining>/<quantity>@<fmt2(price)>
```

Example: `#4 BUY alice INFY 100/100@101.00`. After 30 shares fill: `#4 BUY alice INFY 70/100@101.00`.

7 BuyOrder and SellOrder

Each constructor forwards all four arguments to `Order`.

Class	`side()`	`crosses(otherPrice)`	Meaning
`BuyOrder`	`Side::BUY`	`getPrice() >= otherPrice`	willing to pay at least the ask
`SellOrder`	`Side::SELL`	`getPrice() <= otherPrice`	willing to accept the bid

8 `Trade` Class

A plain value class. The constructor stores all seven fields in the order shown, and the getters return them. Printing it with `operator<<` (a `friend`, returns `os`) writes, with no newline:

```
TRADE <symbol> <quantity>@<fmt2(price)> B#<buyId>:<buyer> S#<sellId>:<seller>
```

Example: `TRADE INFY 30@100.00 B#4:alice S#2:carol`. The buy side always comes first, whichever side was incoming. To turn a trade into a `string` (for logging), use an `ostringstream`: `ostringstream os; os << t; os.str();`

9 `OrderBook` Class

`bids` and `asks` keep orders in **arrival order**; they are never sorted. "Best" is computed by scanning.

9.1 `submit`: the matching algorithm

submit(incoming)

1. If `incoming->getSymbol() != symbol`: `throw ExchangeException("Wrong symbol");`
2. `opposite` = `asks` if incoming is a BUY, else `bids`. `trades` = empty.
3. While `incoming` is not filled:
   (a) Among the orders in `opposite`, a resting order r is **eligible** if `r->getTrader() != incoming->getTrader()` (self-trade prevention) **and** `incoming->crosses(r->getPrice())`.
   (b) Pick the **best** eligible r:
   - resting asks: the **lowest** price; resting bids: the **highest** price;
   - a tie on price goes to the **smallest ID** (it arrived first).

   If no order is eligible, stop the loop.
   (c) `q = min(incoming remaining, r remaining)`; trade price `p = r->getPrice()` (the resting price).
   (d) Append `Trade(symbol, buyId, buyer, sellId, seller, p, q)` to `trades`, with the buy side's ID and trader first.
   (e) `*incoming -= q; *r -= q;` If r is now filled, erase it from `opposite`.
4. If `incoming` still has remaining quantity, append it to its **own** side (`bids` for a buy, `asks` for a sell).
5. Return `trades` (in the order they happened).

Notes. A trader's own resting order is *skipped*, not removed, even when it has the better price. Matching continues with the next eligible order. Because of this, the book can end up "crossed" (a bid $\geq$ an ask from the same trader). That is allowed.

9.2 Worked example

Traders alice, bob and carol, symbol `INFY`. Book shown as `price:remaining(#id)`.

Incoming order	Trades produced	Asks after	Bids after
#1 SELL bob 50@101	none	101:50(#1)	
#2 SELL carol 30@100	none	101:50(#1) 100:30(#2)	
#3 SELL bob 40@100	none	+ 100:40(#3)	
#4 BUY alice 100@101	30@100 with #2 (lowest price, earlier ID), 40@100 with #3, 30@101 with #1	101:20(#1)	
#5 BUY alice 10@99	none ($99 < 101$)	101:20(#1)	99:10(#5)
#6 BUY carol 5@99.5	none		+ 99.5:5(#6)
#7 BUY bob 8@99.5	none ($99.5 < 101$)		+ 99.5:8(#7)
#8 SELL bob 12@99	#7 is bob's own: skipped. 5@99.50 with #6, then 7@99 with #5	101:20(#1)	99.5:8(#7) 99:3(#5)

9.3 `find`, `cancel`, `restingCount`

- `find(id)`: the resting order (bid or ask) with that ID, or `nullptr`.
- `cancel(id, trader)`: removes the resting order with that ID. No such order ⇒ `"Order not found"`. It exists but belongs to another trader ⇒ `"Not your order"` (and it stays).
- `restingCount()`: number of bids plus asks.

9.4 Best prices

`bestBid()` returns the highest bid price; with no bids, `throw ExchangeException("No bids");`. `bestAsk()` returns the lowest ask price; with no asks, `"No asks"`. `spread()` calls `bestBid()` *first*, then `bestAsk()`, and returns ask − bid. These ignore self-trade rules: they just report the top of each side.

9.5 `printDepth`

Prints the book **aggregated by price level**. All orders at the same price form one line showing the sum of their *remaining* quantities and how many orders there are. Asks come first, highest price first, then a `---` line, then bids, highest price first:

```
ASK <fmt2(price)> <total remaining> (<order count>)
...
---
BID <fmt2(price)> <total remaining> (<order count>)
...
```

Example: asks #2 (102, 5 left), #3 (101, 3), #4 (102, 7); bids #5 (99, 4), #6 (100, 6), #7 (99, 1):

```
ASK 102.00 12 (2)
ASK 101.00 3 (1)
---
BID 100.00 6 (1)
BID 99.00 5 (2)
```

Hint: a `map<double, pair<int,int>>` built by scanning, then iterated with `rbegin()`/`rend()`. An empty side prints no lines, but `---` is always printed.

10 Exchange Class

`cash[name]` exists exactly when `name` is registered. `positions[trader][symbol]` is the trader's share count, which may go negative (short selling is allowed). A book is created the first time an order for its symbol is placed.

10.1 registerTrader

Checks in this order, and **throws** (does not log) on failure: already registered ⇒ `"Trader already registered"`; `initialCash < 0` ⇒ `"Invalid cash"`. On success, sets the cash and logs `REGISTER <name> <fmt2(cash)>`.

10.2 placeOrder

Does not throw. The whole body runs inside `try`. Any `ExchangeException` is caught, logged as `REJECT <trader>: <e.what()>`, and the method returns `-1`. On success it returns the new order's ID.
Steps, in this order:

1. Trader not registered ⇒ `"Unknown trader <trader>"` (e.g. `Unknown trader dave`).
2. For a BUY only: if `cash[trader] < price * qty` ⇒ `"Insufficient cash"`. This uses the trader's current cash; nothing is reserved for resting orders.
3. Create the order with `make_shared<BuyOrder>(...)` or `make_shared<SellOrder>(...)`, held in a `shared_ptr<Order>`. Its constructor may throw.
4. Log `ACCEPT <order->describe()>` (before matching, so remaining = quantity).
5. Create the symbol's book if it is missing, then `submit` the order.
6. For each trade in order: the buyer gets `+q` shares and `-p*q` cash; the seller gets `-q` shares and `+p*q` cash. Log the trade's `operator<<` text.
7. Return the order's ID.

Because steps 1–2 run before the order exists, those rejections use up no ID either.

10.3 cancelOrder

Also catches and logs (`REJECT <trader>: <what>`), and returns `false` on failure. An unknown symbol gives `"Order not found"`. Otherwise it calls the book's `cancel`. On success, logs `CANCEL #<id> <trader>` and returns `true`.

10.4 Getters

`getCash(trader)`: unknown trader ⇒ throw `"Unknown trader <trader>"`. `getPosition(trader, symbol)`: same check, then the share count, or `0` if this trader never traded that symbol. `getBook(symbol)`: the book by reference; unknown ⇒ throw `"Unknown symbol <symbol>"`. `getLogCount()`: the logger's count.

10.5 All messages

Thrown by	what()
Order constructor	Invalid quantity Invalid price
Order::operator-=	Invalid fill
OrderBook::submit	Wrong symbol
OrderBook::cancel	Order not found Not your order
OrderBook::bestBid / bestAsk	No bids No asks
Exchange::registerTrader	Trader already registered Invalid cash
Exchange::placeOrder	Unknown trader <name> Insufficient cash
Exchange::getCash / getPosition	Unknown trader <name>
Exchange::getBook	Unknown symbol <symbol>

Table 1: Exact exception messages

11 Driver Program (given, not evaluated)

local_tester/driver.h reads commands until END, and catches ExchangeException as ERR <what>:

Command	Output
REG <name> <cash>	OK
BUY|SELL <trader> <symbol> <price> <qty>	ORDER <id or -1>
CANCEL <trader> <symbol> <id>	CANCEL OK or CANCEL FAIL
BOOK <symbol>	printDepth() of that book
POS <trader> <symbol>	<trader> <symbol> <position> <fmt2(cash)>
END	stops

To run it yourself (from the folder with skeleton.cpp; the log goes to driver_log.txt, which is appended to, so delete it between runs):

```
cp skeleton.cpp local_tester/student.cpp
g++ -std=c++17 local_tester/run_driver.cpp -o drv
rm -f driver_log.txt; ./drv < local_tester/sample1.in; cat driver_log.txt
```